

Agentic AI: Foundations and Open-Source Practice

Day 1 Task — Completed

1. Task Details

Task title	Comparing a Plain Chatbot, a Rule-Based Workflow, and an AI Agent using a course-fee private-data scenario.
Unit	Unit 1: Foundations of AI Agents — Agent = LLM + Tools + Loop
Task type	Individual conceptual analysis based on a private course-fee dataset.
Format required	GitHub repository containing the project code, Output screenshots, and written analysis in Markdown.
Submission	GitHub repository: https://github.com/babitha5327/AI_Fluency_

2. Aim

The chosen private-data scenario is a small course-fee dataset containing CS101 = Rs. 12,000, AI202 = Rs. 18,000, and DS303 = Rs. 15,000. The same course-fee questions were tested using a plain chatbot, a rule-based workflow, and a tool-using AI agent. The experiments show the difference between an LLM-only response, predefined rules, and an LLM + tools + loop.

3. Result

3.1 Explanation of each approach

Plain chatbot: The chatbot mainly uses the LLM to generate responses. In this experiment it did not have access to the private course-fee data or a course-fee lookup tool. Therefore, for questions such as the fee for AI202, it replied that it did not have the fee information. It could still answer a general writing request, such as producing a two-line welcome message. Its main limitation in this scenario is that it cannot reliably retrieve or calculate from the private course-fee data without that data being provided to it.

Rule-based workflow: The workflow uses predefined course-fee data and explicit rules/conditions, with no LLM involved in the decision process. It can retrieve the known fee values, calculate the total after a scholarship, and compare two course fees. For example, it calculated CS101 + AI202 after a 10% scholarship as Rs. 27,000 and identified that DS303 is Rs. 3,000 more expensive than CS101. Its limitation is that a new type of natural-language request requires another predefined rule; for the welcome-message request it returned “No rule for this question.”

AI agent: The agent combines an LLM + Tools + Loop. For the course-fee questions, it selected the **get_course_fee** tool to retrieve private course data and the **calculator** tool for arithmetic. It observed each tool result and continued to the next step until it could answer the question. It successfully handled the fee lookup, multi-course scholarship calculation, and fee comparison, while also answering the general welcome-message request. This demonstrates tool selection, observation, and multi-step execution.

Observed agent examples: AI202 → Rs. 18,000; CS101 + AI202 after 10% scholarship → Rs. 27,000; DS303 vs CS101 → Rs. 3,000 difference. The Day 1 challenge also calculated the same two-course total after a 20% scholarship as Rs. 24,000.

3.2 Comparison table

Basis for comparison	Plain chatbot	Rule-based workflow	AI agent
Flexibility	High for natural-language generation, but limited by missing private data.	Low to medium; depends on predefined rules.	High; can adapt to different requests and choose tools.
Decision-making	Generates an LLM response; no tool-based decision process.	Follows fixed conditions and steps.	LLM reasons about the task and selects the next tool/action.
Tool usage	None in this implementation.	Uses predefined program logic/calculations, not an LLM tool loop.	Uses get_course_fee and calculator tools.
Private-data access	No access in this implementation.	Direct access to hardcoded course-fee data.	Accesses private course data through get_course_fee .
Multi-step task handling	Limited; it lacked the required fee information.	Works when the exact sequence is predefined.	Handles retrieval → calculation → observation → final answer.

Basis for comparison	Plain chatbot	Rule-based workflow	AI agent
Automation	Low for private-data tasks without connected tools/data.	High for known, repetitive cases.	High; can automate multi-step tasks using selected tools.
Reliability	Weak for this private-data scenario because it lacked the data.	High for explicitly covered cases and rules.	Good when tools and tool calls are correctly configured; depends on model/tool integration.

3.3 Suitability analysis

For this course-fee scenario, the AI agent is the most suitable when the user may ask different questions that require both private-data access and multi-step calculations. The agent can retrieve the required course fees, use the calculator, observe the results, and then produce the answer. The rule-based workflow is also effective for fixed, repetitive questions and is predictable, but it needs explicit rules for each supported question type. The plain chatbot is useful for general-language responses but, in the tested implementation, could not answer questions requiring the private course-fee data.

3.4 Conclusion

A plain chatbot is appropriate when the main requirement is natural-language conversation or generation and no private-data lookup or external action is required. A rule-based workflow is appropriate for predictable, repetitive tasks where the steps and conditions are known in advance and consistent behavior is important. An AI agent is appropriate when a task requires flexible reasoning, access to tools or private data, and multiple steps that may need to be selected dynamically. The Day 1 experiment demonstrates the progression from LLM-only responses, to fixed rules, to an LLM + Tools + Loop agent.

4. Submission Guidelines — Completed Status

1. Project code for chatbot.py, workflow.py, tools.py, agent.py, challenge.py, challenge_official.py, check_setup.py, config.py, and requirements.txt has been prepared.
2. The GitHub repository AI_Fluency_ has been created and the project files were uploaded. The repository also contains .gitignore so that local secrets and environment folders are excluded.
3. The required written analysis can be saved as analysis.md in the repository. The content above is the completed analysis.
4. GitHub repository: https://github.com/babitha5327/AI_Fluency_

Important: The original task requires output screenshots of all three systems inside an Output folder. Add the screenshots of chatbot.py, workflow.py, and agent.py outputs to that folder before final submission.